

Lógica Computacional 2024/2025 - TP4

Grupo 6

- Cláudia Faria, a105531
- Patrícia Bastos, a102502

```
In [ ]: %%capture
!yes | pip install pysmt
!yes | pysmt-install --z3
```

```
In [ ]: from pysmt.shortcuts import *
from pysmt.typing import *
```

Exercício 2

Enunciado

Este exercício é dirigido à prova de correção do algoritmo estendido de Euclides apresentado no trabalho TP3

- Construa a asserção lógica que representa a pós-condição do algoritmo. Note que a definição da função $\text{gcd}(a, b) \equiv \min\{r > 0 \mid \exists s, t. r = a * s + b * t\}$.
- Usando a metodologia do comando **havoc** para o ciclo, escreva o programa na linguagem dos comandos anotados (LPA). Codifique a pós-condição do algoritmo com um comando **assert**.
- Construa codificações do programa LPA através de transformadores de predicados "strongest post-condition" e prove a correção do programa LPA.

Algoritmo estendido de Euclides

O algoritmo estendido de Euclides (EXA) aceita dois inteiros constantes $a, b > 0$ e devolve inteiros r, s, t tais que $a * s + b * t = r$ e $r = \text{gcd}(a, b)$.

Para além das variáveis r, s, t o código requer 3 variáveis adicionais r', s', t' que representam os valores de r, s, t no "próximo estado".

```
INPUT  a, b
assume a > 0 and b > 0
r, r', s, s', t, t' = a, b, 1, 0, 0, 1

0: while r' != 0
1:   q = r div r'
2:   r, r', s, s', t, t' = r', r - q * r', s', s - q * s',
   t', t - q * t'
```

3: stop

OUTPUT r, s, t

Para simplificar o uso de r', s' e t' nas resoluções que se seguem, estes serão referidos como rl, sl e tl , respetivamente.

a. Construa a asserção lógica que representa a pós-condição do algoritmo. Note que a definição da função gcd é $\text{gcd}(a, b) \equiv \min\{r > 0 \mid \exists s, t. r = a * s + b * t\}$.

```
r = a*s + b*t
and
r > 0
and
forall x,y,z . (x = a*y + b*z and x > 0) -> r <= x
```

b. Usando a metodologia do comando **havoc** para o ciclo, escreva o programa na linguagem dos comandos anotados (LPA). Codifique a pós-condição do algoritmo com um comando **assert**.

```
assume a > 0 and b > 0;
r, r', s, s', t, t' <- a, b, 1, 0, 0, 1;
havoc r, r', s, s', t, t';

(
  (
    // Corpo do loop
    assume r' != 0;
    q <- r div r';
    r, r', s, s', t, t' <- r', r - q * r', s', s - q * s',
    t', t - q * t';
    assume False
  )
)
||
(
  (
    // Saída do loop
    assume r' = 0;
  )
  assert (
    r = a * s + b * t and           // Combinação linear
    r > 0 and                       // Positividade
    forall x, y, z .                // Propriedade de
minimalidade                       (x = a * y + b * z and x > 0) -> r <= x
  )
)
```

```
);
```

c. Construa codificações do programa LPA através de transformadores de predicados "strongest post-condition" e prove a correção do programa LPA.

De forma a provar a correção do programa LPA é usada a função `prove` fornecida nas aulas práticas.

```
In [ ]: def prove(f):  
  
    with Solver(name="z3") as s:  
        s.add_assertion(Not(f))  
        if s.solve():  
            print("Failed to prove.")  
        else:  
            print("Proved.")
```

Explicação:

- *preCondicao* garante que o estado inicial do programa está corretamente configurado

```
assume a > 0 and b > 0;  
r, r', s, s', t, t' <- a, b, 1, 0, 0, 1;
```

- *invariante* garante que estado intermediário do programa durante o loop é válido

```
r' > 0 and  
r = a * s + b * t and  
r' = a * s' + b * t'
```

em que:

- `r' > 0` garante que o cálculo do GCD ainda está em progresso
- `r = a * s + b * t` garante que o valor atual de `r` é uma representação válida
- `r' = a * s' + b * t'` garante que o próximo valor de `r'` é uma representação válida.
- *loop* atualiza as variáveis

```
assume r' != 0;  
q <- r div r';  
r, r', s, s', t, t' <- r', r - q * r', s', s - q * s', t',  
t - q * t';
```

- *posCondicao* define o comportamento esperado no final do programa (como definido na alínea a)

```

r = a*s + b*t
and
r > 0
and
forall x,y,z . (x = a*y + b*z and x > 0) -> r <= x

```

- *vc* são as condições de verificação
- Inicialização do Invariante

```

vc = And(preCondicao, Exists([r, rl, s, sl, t, tl, q],
invariante))

```

Verifica se o invariante pode ser inicializado corretamente a partir da pré-condição.

O Exists é usado para demonstrar que é possível encontrar ao menos um conjunto de valores para as variáveis que torna o invariante verdadeiro, dado o estado inicial definido pela pré condição.

- Preservação do invariante:

```

vc = Or(And(vc, loop), And(vc, Equals(rl, Int(0))))

```

Verifica se o invariante é preservado durante cada iteração do loop e a sua validade ao terminar.

- Verificação final:

```

Implies(vc,posCondicao)

```

Garante que ao terminar o programa a pós condição é satisfeita.

```

In [ ]: a = Symbol('a', INT)
b = Symbol('b', INT)
r = Symbol('r', INT)
rl = Symbol('rl', INT)
s = Symbol('s', INT)
sl = Symbol('sl', INT)
t = Symbol('t', INT)
tl = Symbol('tl', INT)
q = Symbol('q', INT)
x = Symbol('x', INT)
y = Symbol('y', INT)
z = Symbol('z', INT)

preCondicao = And(
    GT(a, Int(0)), GT(b, Int(0)),
    Equals(r, a),
    Equals(rl, b),
    Equals(s, Int(1)),
    Equals(sl, Int(0)),

```

```

    Equals(t, Int(0)),
    Equals(tl, Int(1))
)

invariante = And(
    GT(rl, Int(0)),
    Equals(r, Plus(Times(a, s), Times(b, t))),
    Equals(rl, Plus(Times(a, sl), Times(b, tl)))
)

vc = And(preCondicao, Exists([r, rl, s, sl, t, tl, q], invariante))

loop = And(
    Not(Equals(rl, Int(0))),
    Equals(q, Div(r, rl)),
    Equals(r, rl),
    Equals(rl, Minus(r, Times(q, rl))),
    Equals(s, sl),
    Equals(sl, Minus(s, Times(q, sl))),
    Equals(t, tl),
    Equals(tl, Minus(t, Times(q, tl)))
)

vc = Or(And(vc, loop), And(vc, Equals(rl, Int(0))))

posCondicao = And(
    GT(r, Int(0)),
    Equals(r, Plus(Times(a, s), Times(b, t))),
    ForAll([x, y, z],
        Implies(
            And(GT(x, Int(0)),
                Equals(x, Plus(Times(a, y), Times(b, z)))),
            LE(r, x)
        )
    )
)

prove(Implies(vc, posCondicao))

```

Proved.